

Malware Detection from Network Flow Data using Flow-Level Machine Learning

Youbeen Oh

December 12, 2025

1 Dataset

I use the Malware Detection dataset from the pcapML / nPrint benchmark suite. The dataset consists of raw network traffic in `.pcapng` format with metadata indicating whether each flow is associated with malware or benign activity.

After processing, I obtain a flow-level table with 498,446 samples. Among these, 369,325 flows are labeled as malware and 129,121 as benign, which corresponds to a moderately imbalanced binary classification problem.

2 Goal

The goal of this project is to build machine learning models that can distinguish malicious from benign network flows using only flow-level statistics. This setting is realistic because modern traffic is frequently encrypted, which makes it difficult or impossible to rely on payload inspection. Instead, the model must learn to detect attacks from patterns in timing, sizes, and other header-level behavior.

3 Label Construction with pcapML

I use the pcapML Python interface to iterate over each traffic sample in the `.pcapng` file. Each sample contains metadata such as:

```
malware_adload
benign_benign
malware_emotet
...
```

To obtain a simple binary label, I assigned ground-truth target label $\in \{0, 1\}$ for both supervised learning and for evaluating the unsupervised baseline.

The raw labels are stored in the `metadata` field of each pcapML sample. On the Ubuntu environment (outside the notebook), I used a small Python script to convert these metadata strings into binary labels. Conceptually, the logic was as follows:

```
from pcapml_fe import sampler
```

```

y = []
for sample in sampler("traffic.pcapng"):
    meta = sample.metadata
    # make sure we have a lowercase string
    if isinstance(meta, bytes):
        meta = meta.decode("utf-8", errors="ignore")
    meta = str(meta).strip().lower()

    label = 1 if "malware" in meta else 0
    y.append(label)

```

4 Feature Engineering

Each flow is converted into 11 numerical summary features:

- Packet count
- Total bytes
- Packet size: mean, median, min, max, std
- Flow duration
- Inter-arrival time: mean, std, max

The feature extraction function used is shown below:

```

import numpy as np

def build_flow_features(sample):
    """
    Turn one pcapML sample into a fixed-length
    feature vector plus a binary label.
    """

    pkts = getattr(sample, "packets", [])
    if not pkts:
        return None, None

    # packet sizes and timestamps
    sizes = np.array([len(p.raw_bytes) for p in pkts], dtype=float)
    times = np.array([p.ts for p in pkts], dtype=float)

    # basic volume
    pkt_count = float(sizes.size)
    bytes_total = float(sizes.sum())

    # size statistics
    size_mean = float(sizes.mean())

```

```

size_median = float(np.median(sizes))
size_min    = float(sizes.min())
size_max    = float(sizes.max())
size_std    = float(sizes.std())

# timing statistics
if times.size > 1:
    duration = float(times.max() - times.min())
    iats      = np.diff(np.sort(times))
    iat_mean  = float(iats.mean())
    iat_std   = float(iats.std())
    iat_max   = float(iats.max())
else:
    duration = 0.0
    iat_mean = 0.0
    iat_std  = 0.0
    iat_max  = 0.0

features = [
    pkt_count,
    bytes_total,
    size_mean,
    size_median,
    size_min,
    size_max,
    size_std,
    duration,
    iat_mean,
    iat_std,
    iat_max,
]

meta = str(sample.metadata).lower()
label = 1 if "malware" in meta else 0

return features, label

```

Instead of operating directly on raw packets, I construct flow-level summary features. For each flow I compute:

- **Volume features:** packet count (`pkt_count`) and total bytes (`bytes_sum`).
- **Packet size statistics:** mean, median, minimum, maximum, and standard deviation of packet sizes.
- **Temporal features:** flow duration and inter-arrival time (IAT) statistics including mean, standard deviation, and maximum.

In total, this yields eleven numerical features per flow, stored in the file `traffic_flow_features_binary_custom`

These features are designed to capture how heavy the flow is, how bursty it is, and how regular or irregular the timing looks, all of which are plausible signals for malware behavior.

5 Unsupervised Learning: One-Class SVM

Before training supervised models, I first ask whether malware flows can be detected as anomalies, using only benign traffic for training. This corresponds to a realistic scenario where only a small subset of flows has been manually labeled, or where the system is deployed in an environment that is assumed to be mostly clean.

5.1 Method

```
from sklearn.decomposition import PCA
from sklearn.svm import OneClassSVM
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
Xb = sc.fit_transform(X_benign)

pca = PCA(n_components=3)
Xb_pca = pca.fit_transform(Xb)

oc = OneClassSVM(kernel="rbf", nu=0.1)
oc.fit(Xb_pca)
```

I train a One-Class SVM (OCSVM) with an RBF kernel on benign flows only. Let X_{benign} denote the subset of flow features with label 0 in the training split. After standardizing each feature to have zero mean and unit variance, I fit the model on X_{benign} using parameter $\nu = 0.1$, which roughly controls the expected fraction of anomalies. At test time, I apply the trained OCSVM to the mixed test set containing both benign and malware flows.

The OCSVM outputs $+1$ for samples that are considered normal and -1 for samples that are considered anomalous. For evaluation, I map these predictions to binary labels by treating anomalous flows as predicted malware:

5.2 Results

On the held-out test set, the OCSVM produces the following confusion matrix:

$$\begin{bmatrix} 34835 & 3901 \\ 73453 & 37345 \end{bmatrix},$$

where rows correspond to the true class (benign, malware) and columns to the predicted class (benign, malware). In other words, the model correctly classifies 34,835 benign flows and 37,345 malware flows, but also misclassifies 3,901 benign flows as malware and 73,453 malware flows as benign.

The balanced accuracy is

$$\text{Balanced Accuracy} = 0.6182,$$

which is above random guessing and indicates that the model is learning some structure in the data, despite being trained only on benign samples.

Table 1 summarizes the precision, recall, and F1-score per class.

Table 1: Classification report for OCSVM anomaly detection on the test set.

Class	Precision	Recall	F1-score	Support
Benign (0)	0.3217	0.8993	0.4739	38,736
Malware (1)	0.9054	0.3371	0.4912	110,798
Overall accuracy	0.4827			
Macro average	0.6136	0.6182	0.4826	149,534
Weighted average	0.7542	0.4827	0.4867	149,534

5.3 Discussion

The OCSVM does a reasonably good job of recognizing benign traffic: the recall for the benign class is about 0.90. This means that once the model has seen enough clean flows, it can describe the normal region of feature space fairly well.

However, the recall for malware is only about 0.34, and the overall accuracy is 0.48. Many malware flows are not flagged as anomalies, which suggests that a large fraction of malicious traffic appears relatively similar to benign flows in the chosen feature space. In other words, an unsupervised detector based only on benign data captures some differences between the two classes, but it is not sufficient on its own for high-quality malware detection.

This unsupervised result is still useful: it confirms that there is some separability between benign and malicious flows, and it provides a baseline for what can be achieved without labels. In the next section, I move to fully supervised models that use the same flow-level features but exploit the labels directly.

6 Supervised Learning

To establish stronger baselines and take advantage of the available labels, I train several supervised models on the engineered flow-level features. Unlike the OCSVM anomaly detector, these models directly learn the boundary between malware and benign traffic. All models are evaluated on a held-out test set using accuracy, balanced accuracy, F1-score, and ROC AUC.

6.1 Logistic Regression

I begin with Logistic Regression as a simple linear baseline. Because the feature set is relatively low-dimensional and numerical, logistic regression provides a useful reference model for understanding separability.

```
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression(max_iter=200)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

The performance metrics are:

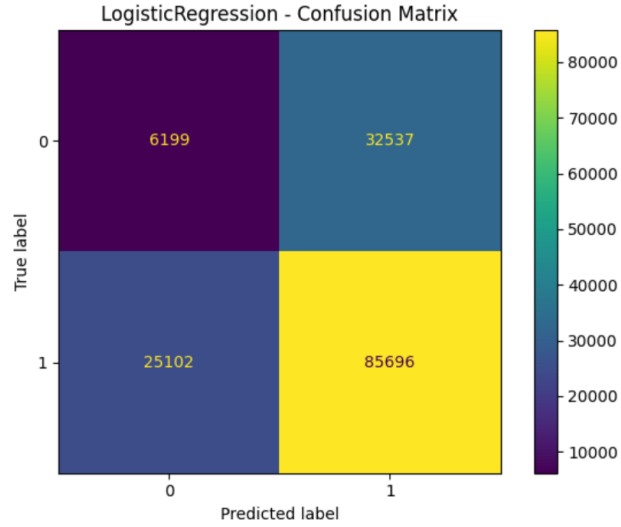


Figure 1: Confusion Matrix for Logistic Regression

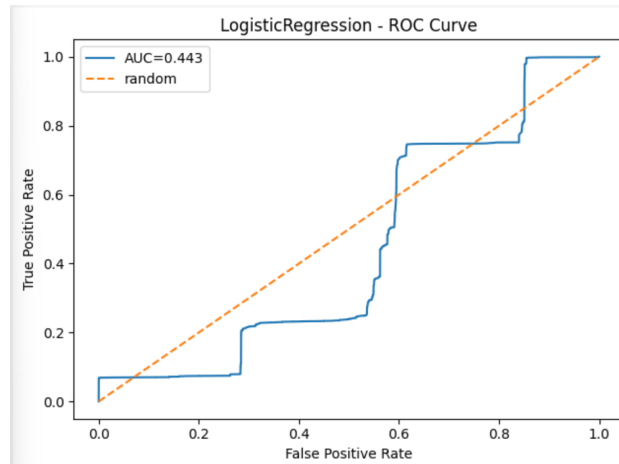


Figure 2: ROC Curve for Logistic Regression

```
===== LogisticRegression =====
Accuracy      : 0.6145
Balanced Accuracy : 0.4667
F1-score      : 0.7483
ROC AUC       : 0.4435
```

While the F1-score is moderate, the balanced accuracy and ROC AUC indicate weak predictive power. This confirms that the relationship between features and class label is not linearly separable, and motivates moving to nonlinear models.

6.2 Random Forest

Random Forest serves as a strong non-linear model capable of capturing interactions and irregular patterns in flow behavior. It is also fast to train and highly interpretable in terms of feature importance.

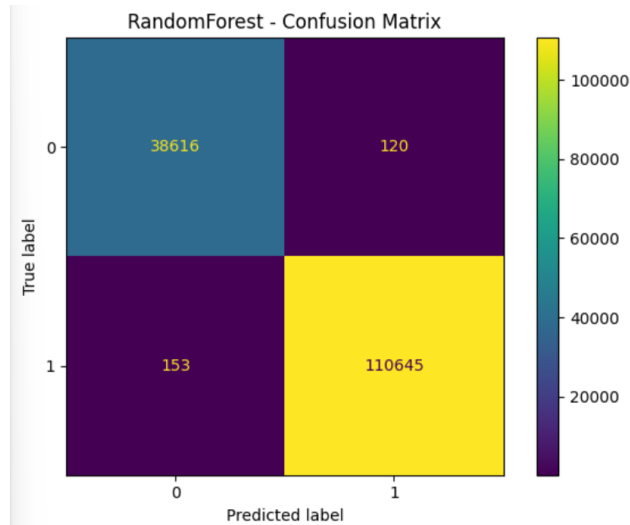


Figure 3: Confusion Matrix for Random Forest

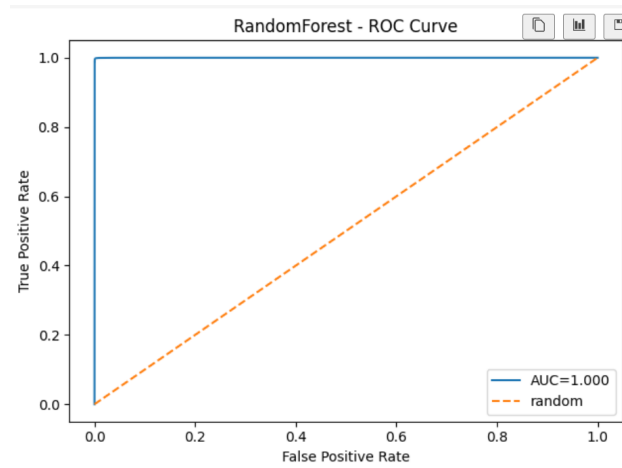


Figure 4: ROC Curve for Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=200)
rf.fit(X_train, y_train)

===== RandomForest =====
Accuracy          : 0.9982
Balanced Accuracy : 0.9978
F1-score          : 0.9988
ROC AUC           : 0.9999
```

This represents a dramatic improvement over logistic regression. The model is able to distinguish malware from benign flows almost perfectly, suggesting that the engineered feature set is highly informative.

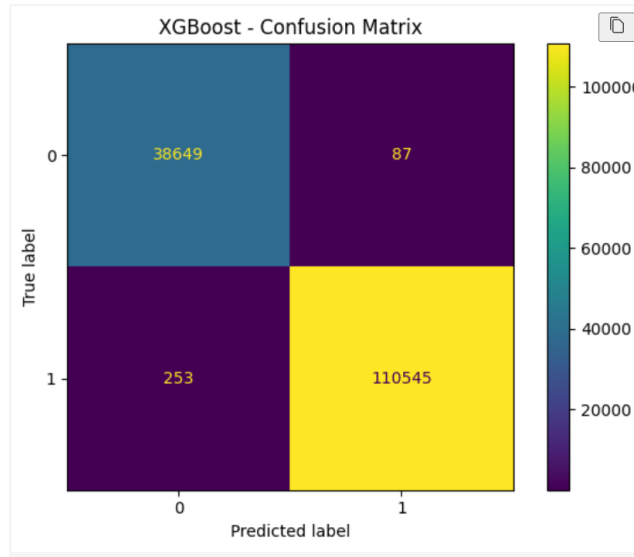


Figure 5: Confusion Matrix for XGBoost

6.3 XGBoost

XGBoost is known for its efficiency and strong predictive performance on tabular data. It also provides regularization and fast training, which makes it attractive for real-world deployment.

```
import xgboost as xgb

xgbc = xgb.XGBClassifier(tree_method="hist")
xgbc.fit(X_train, y_train)

===== XGBoost =====
Accuracy          : 0.9977
Balanced Accuracy : 0.9977
F1-score          : 0.9985
ROC AUC           : 0.9999
```

The model achieves nearly the same performance as Random Forest, with a slightly faster training pipeline. This suggests that both models are able to leverage the non-linear structure in the flow features.

6.4 LightGBM

LightGBM is optimized for speed and memory efficiency, using histogram-based gradient boosting. Given the size of this dataset (nearly half a million flows), LightGBM provides an excellent trade-off.

```
import lightgbm as lgb

lgbm = lgb.LGBMClassifier()
lgbm.fit(X_train, y_train)
```

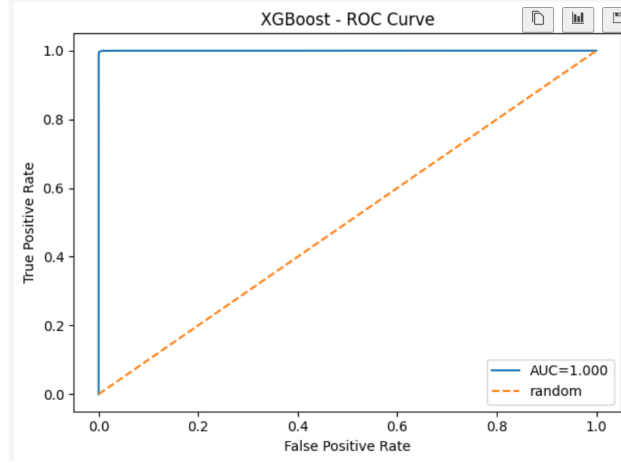



Figure 6: ROC Curve for XGBoost

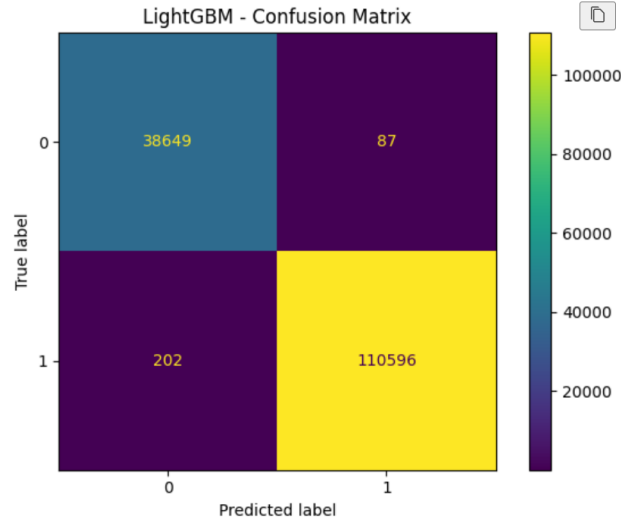


Figure 7: Confusion Matrix for LightBGM

===== LightGBM =====

Accuracy : 0.9981
 Balanced Accuracy : 0.9980
 F1-score : 0.9987
 ROC AUC : 1.0000

LightGBM matches the performance of the other tree-based models while offering the fastest training time among them. For network security applications, where rapid retraining may be necessary, this efficiency is a practical advantage.

7 Results

Table 2 summarizes all supervised model performances.

All three tree-based models achieve outstanding performance, effectively solving the binary

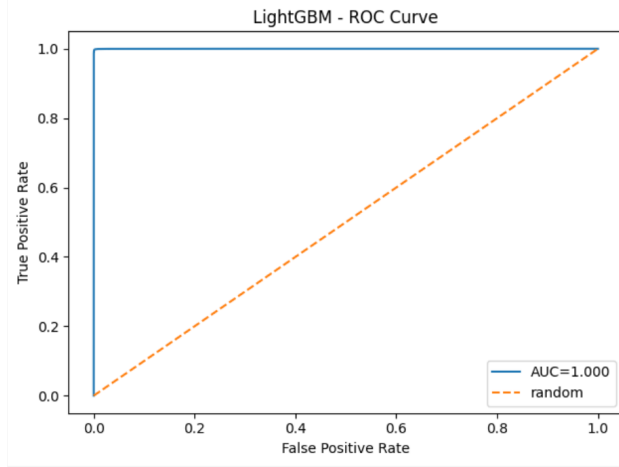


Figure 8: ROC Curve for LightBGM

Table 2: Summary of supervised model performance.

Model	Accuracy	Balanced Acc.	F1-score	ROC AUC
Logistic Regression	0.6145	0.4667	0.7483	0.4435
Random Forest	0.9982	0.9978	0.9988	0.9999
XGBoost	0.9977	0.9977	0.9985	0.9999
LightGBM	0.9981	0.9980	0.9987	1.0000

classification problem on this dataset. This reinforces the idea that flow-level statistical features are highly predictive of malicious activity.

8 Final Comments

This project reproduces the malware detection benchmark from the pcapML suite and extends it by constructing a custom feature set. Several key observations emerge:

- Flow-level statistics (packet size distribution, timing irregularity, and burstiness) are highly discriminative for malware traffic.
- Even without payload inspection, machine learning models can reliably detect malicious behavior using metadata alone.
- Unsupervised anomaly detection (OCSVM) captures part of the structure but misses many malware cases, highlighting the need for supervised learning in high-stakes environments.
- Tree-based models (Random Forest, XGBoost, LightGBM) achieve near-perfect accuracy and robustness, making them strong candidates for real-world deployment.

Overall, the combination of domain-guided feature engineering and modern tree-based algorithms results in highly effective malware detection. Future work may include testing generalization across datasets, exploring multi-class malware family classification, and experimenting with representation learning from raw packet sequences.